

Transfer Function Estimation

O.A.H

May 2026

Regression

Problem

We can use non-linear least square to choose the optimal set of pole and zeros by minimising

$$\min || Y(s) - \frac{\Pi(s - z_i)}{\Pi(s - p_i)} X(s) ||_2^2 \quad (1)$$

that is the mean squared error between the observed spectra and the current filter coefficients.

We define the scalar loss function

$$J(\mathbf{m}) = || \frac{1}{N} \sum_{i=0}^N Y_i - \mathbf{g}(\mathbf{m}, X_i) || \quad (2)$$

$$\mathbf{g}(\mathbf{m}, X_i) = \frac{\Pi(s - m_i)}{\Pi(s - m_{i+n_z})} X_i(s) \quad (3)$$

So are attempting to minimise the total loss over all of the N observations.

Parametrisation

Specifically we regress for filter coefficients in units of rad s^{-1} and output the poles and zeros in Cartesian coordinates ($a+bi$). However, for the regression itself we parametrise them in polar coordinates ($r \exp(i\theta)$) as is easier to enforce the required bounds.

We only attempt to find poles and zeros in the upper half of the complex plane, as they must be symmetric over the real axis. Furthermore, for linear phase filters we also calculate the reciprocal conjugates to as the coefficients must be antisymmetric over the unit circle

Data Transform

The observed spectrograms are transformed according to

$$\hat{Y} = \log_{10}(|Y| + \epsilon) \quad (4)$$

in order to remove the order of magnitude scaling in the data.

Bounds

Currently, the regression algorithm only supports FIR filters, so only the zeros need to be located as all poles are at the origin. For a minimum phase filter with N zeros, we enforce the poles to be within the unit circle ($r < 1$) and in the upper complex plane ($0 \leq \theta \leq \pi$). Ultimately, we output $2N$ zeros (the zeros we found and their complex conjugates).

For a linear phase filter we enforce that $1/3 < r < 3$ and $0 \leq \theta \leq \pi$ and return $4N$ zeros being $z, z^*, \frac{1}{z}, \frac{1}{z^*}$ for any zero z .

Algorithm

We use L-BFGS-B (Low memory-Broyden-Fletcher-Goldfarb-Shanno-Bounded) to optimise. We simply use the `scipy.optimize.minimize` implementation. The initial model is set to be a semi circle of radius 0.7 in the upper half of the complex plane. The frequency response (the fraction in `g`) is calculated using `scipy.signal.freqz` and an option does exist to swap to a custom implementation if needed.

Data Collection

The data collections is split into 5 stages that are designed to cover all the important parts of the spectrum. The details and reasoning for each stage is given below, assuming a datalogger sampling rate of f Hz.

1. log-spaced between 0.5 Hz and $0.6f$ Hz. Designed to mainly cover the lower end
2. Random, uniform samples between 0.5Hz and $1.5f$, mainly to aid in regularising it and testing for any aliasing
3. Harmonics of f (i.e. $\{2f, f, 0.5f, 0.25f\}$) to sample any odd aliasing effects
4. Constantly spaced samples between $0.4f$ and $0.6f$ to get the drop on the top end.
5. Uniform samples between $0.2f$ and $0.6f$ to regularise the top end and pack out the number of samples